
-----CICD-----

◆ 1. GitHub Actions

🔍 Summary:

GitHub Actions is a CI/CD platform built into GitHub. You define workflows as `.yaml` files that trigger on events like push, PR, or schedule.

✅ Example:

```
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm test
```

💬 Interview Q&A:

Q1: What are GitHub Actions used for? A1: Automating workflows like test, build, deploy, or releasing code when GitHub events occur.

Q2: What is a GitHub Action vs a Workflow? A2:

- **Workflow:** Entire pipeline (`.github/workflows/*.yaml`)
- **Action:** A reusable step (e.g., `actions/checkout`) inside the workflow

Q3: How do you trigger a GitHub Action only on changes to a specific folder? A3:

```
on:
  push:
    paths:
      - 'api/**'
```

Q4: Can GitHub Actions deploy to AWS/GCP? A4:  Yes, via official and community actions (e.g., `aws-actions/configure-aws-credentials` , `google-github-actions/deploy-appengine`)

◆ 2. Jenkins Pipelines

Summary:

Jenkins Pipelines are code-defined CI/CD processes using **Declarative** or **Scripted** syntax in `Jenkinsfile` .

Example:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'npm install'
      }
    }
    stage('Test') {
      steps {
        sh 'npm test'
      }
    }
  }
}
```

Interview Q&A:

Q1: Declarative vs Scripted pipeline — difference? A1:

- **Declarative:** simpler, structured
 - **Scripted:** flexible, Groovy-based logic
-

Q2: What is a Jenkinsfile? A2: A version-controlled file that defines a Jenkins pipeline as code.

Q3: How do you parameterize a Jenkins pipeline? A3:

```
parameters {
  string(name: 'ENV', defaultValue: 'staging')
}
```

Q4: How do you share steps across Jenkins pipelines? A4:

- Use shared libraries in `vars/`
 - Create reusable Groovy functions
-

3. CI/CD Tools

Summary:

CI/CD tools automate the process of **building, testing, and deploying** code. They vary by integration depth, ecosystem support, and scalability.

Common Tools:

Tool	Notes
GitHub Actions	GitHub-native, YAML-based
Jenkins	Open source, highly flexible

Tool	Notes
GitLab CI	GitLab-native
CircleCI	Cloud-native, simple YAML
Bitbucket Pipelines	Native to Bitbucket

Interview Q&A:

Q1: What factors to consider when choosing a CI/CD tool? A1:

- Code hosting platform
 - Team size and scale
 - Deployment targets (AWS, K8s, etc.)
 - Self-hosted vs cloud-based
 - Ease of use and extensibility
-

Q2: GitHub Actions vs Jenkins — pros/cons? A2:

- **GitHub Actions:** easier for GitHub users, fast to adopt
 - **Jenkins:** customizable, plugin-rich, but needs hosting
-

Q3: Can CI/CD be fully serverless? A3:  Yes, using tools like GitHub Actions, CircleCI, or GitLab CI — they abstract the runner infrastructure.

Q4: How do these tools handle secrets? A4: Most provide **encrypted secret management** (e.g., GitHub Secrets, Jenkins Credentials).

4. Deployment Strategies

Summary:

Deployment strategy defines how new versions are rolled out to users. It balances **speed, reliability,** and risk.

✓ Key Strategies:

Strategy	Use Case
Recreate	Tear down & redeploy (simple)
Rolling	Update instances in batches
Blue-Green	Switch traffic to new env
Canary	Gradual % rollout
A/B Testing	Experiment with features

💬 Interview Q&A:

Q1: What is Blue-Green deployment? A1: Two environments (blue = live, green = new). Switch traffic only when green is fully ready.

Q2: Difference between Canary and A/B testing? A2:

- **Canary** = gradual rollout to monitor stability
 - **A/B** = test variants for business/UX experiments
-

Q3: When would you use Recreate? A3: For non-critical apps or dev environments where downtime is acceptable.

Q4: Which strategy offers the best rollback option? A4: Blue-Green — you just switch traffic back to the old environment.

◆ 5. Bitbucket Pipelines

Summary:

Bitbucket Pipelines is Bitbucket's built-in CI/CD tool. It uses a YAML file (`bitbucket-pipelines.yml`) to define automated steps like build, test, and deploy.

Example:

```
pipelines:
  default:
    - step:
        name: Build & Test
        image: node:18
        caches:
          - node
        script:
          - npm install
          - npm test
```

Interview Q&A:

Q1: What is the file name and location for Bitbucket Pipelines? A1: `.bitbucket-pipelines.yml` in the root of the repo.

Q2: What is a pipeline step? A2: A set of commands executed in a clean container — like `npm install` , `pytest` , etc.

Q3: Can Bitbucket Pipelines deploy to AWS/GCP? A3:  Yes. You can use CLI tools, custom scripts, or Docker images with `aws-cli` , `gcloud` , etc.

Q4: How do you trigger pipelines manually or by branch? A4: Using `custom:` pipelines or specifying `branches:` in the config.

6. Pipeline as Code

Summary:

Pipeline as Code means storing the CI/CD pipeline logic **inside your codebase**, version-controlled alongside your app.

Examples:

- **GitHub Actions:** `.github/workflows/*.yaml`
 - **Jenkins:** `Jenkinsfile`
 - **Bitbucket:** `.bitbucket-pipelines.yaml`
 - **GitLab:** `.gitlab-ci.yaml`
-

Interview Q&A:

Q1: What are the benefits of Pipeline as Code? A1:

- Version-controlled
 - Reproducible
 - Easier reviews & rollbacks
 - Consistency across environments
-

Q2: How do you manage complex pipelines in code? A2:

- Use YAML anchors/extends (GitLab, Bitbucket)
 - Split into reusable files or templates
 - In Jenkins, use shared libraries or multibranch pipelines
-

Q3: Is pipeline as code secure? A3: Only if sensitive data (e.g., secrets) are **not hardcoded** — use secrets managers or CI-provided encrypted vars.

Q4: Can pipeline definitions support conditional logic? A4:  Yes — most CI tools support `if` , `when` , or scripting.

◆ 7. Environments

Summary:

Environments in CI/CD represent target stages like `dev` , `staging` , or `prod` — each may have specific configs, credentials, or deployment logic.

Example:

```
deploy:
  stage: deploy
  environment: production
  script: ./deploy.sh --env=prod
```

Interview Q&A:

Q1: What are environments used for in CI/CD? A1: To define target stages for deployment and manage behavior based on that stage.

Q2: Can you enforce rules per environment? A2:  Yes — e.g., approval gates for `prod` , auto-deploy for `dev` .

Q3: How are environment-specific secrets handled? A3: Through CI secrets configuration tied to environments (e.g., GitHub Actions Environments, Jenkins credentials per env).

Q4: How do you manage consistency across environments? A4:

- Use environment variables
 - Template configurations
 - Infra as code (e.g., Terraform workspaces, Helm values)
-

◆ 8. Build & Test Steps

Summary:

These are the core of your pipeline: **build** converts code to artifacts (binaries, images), and **test** validates correctness.

Example:

```
steps:
  - name: Build
    script: npm run build
  - name: Test
    script: npm test
```

Interview Q&A:

Q1: What's the typical order of CI steps? A1:

1. Checkout
 2. Install dependencies
 3. Build
 4. Test
 5. Lint/static analysis
 6. Package
 7. Deploy
-

Q2: Should tests run in all environments? A2:  Yes, but you may run **unit tests** in `dev`, and **integration/end-to-end** in `staging` or `pre-prod`.

Q3: How to reduce build/test time? A3:

- Use caching (e.g., npm, Docker layers)
- Run tests in parallel

- Split into multiple pipeline steps or runners
-

Q4: What types of tests should be included in CI? A4:

- Unit tests
 - Integration tests
 - Static analysis (lint, type check)
 - Optional: performance, security scanning
-
-

◆ 9. Versioning

Summary:

Versioning in CI/CD refers to automatically tagging builds/releases with identifiable and traceable semantic versions (`v1.2.3`) or **build numbers**.

Example:

```
- name: Set version
  run: echo "RELEASE_TAG=v1.0.$GITHUB_RUN_NUMBER" >> $GITHUB_ENV
```

Interview Q&A:

Q1: What are common versioning strategies? A1:

- Semantic Versioning (SemVer): `MAJOR.MINOR.PATCH`
 - Commit-based: `v1.0.123-gabcdef`
 - Date-based: `2024.06.09`
-

Q2: How do you version Docker images in CI/CD? A2: Use build context (commit SHA, tag, or pipeline ID) and apply:

```
docker build -t myapp:$VERSION .
```

Q3: Why is consistent versioning important? A3:

- Enables rollback
- Ensures traceability
- Supports cache control and release tracking

Q4: What tools can help manage versioning? A4:

- `git describe`
- `semantic-release`
- CI/CD system variables (e.g., `$BITBUCKET_BUILD_NUMBER` , `$GITHUB_SHA`)

◆ 10. Deployment Targets

Summary:

Deployment targets are the environments or services where artifacts are deployed: e.g., **EC2**, **ECS**, **Kubernetes**, **Cloud Functions**, or **bare-metal servers**.

Examples:

```
# Deploy to AWS S3
- run: aws s3 sync ./build s3://my-bucket --delete

# Deploy to Kubernetes
- run: kubectl apply -f k8s/deployment.yaml
```

Interview Q&A:

Q1: What are common deployment targets? A1:

- Servers (EC2, SSH)
 - Containers (ECS, EKS, GKE, Docker Swarm)
 - Serverless (Lambda, Cloud Functions)
 - Static (S3, Firebase Hosting)
-

Q2: How does deployment differ by target? A2: Each target has different requirements (e.g., container images for K8s, ZIPs for Lambda), and the CI must prepare the artifacts accordingly.

Q3: How do you abstract deployment logic? A3: Use scripts, makefiles, or deployment templates (e.g., Helm for K8s, Terraform, Ansible, or CD tools like ArgoCD).

Q4: How do you control deployment access in CI/CD? A4: Use cloud IAM roles, scoped API tokens, or CI secret stores (e.g., GitHub OIDC tokens for AWS auth).

◆ 11. Rollbacks

🔍 Summary:

A **rollback** reverts a deployment when something goes wrong. It's often handled via tagging, versioned artifacts, or environment switching (e.g., Blue-Green).

✅ Examples:

```
kubectl rollout undo deployment/myapp  
aws lambda update-function-code --function-name myfunc --s3-key previous.zip
```

💬 Interview Q&A:

Q1: What's the difference between rollback and redeploy? A1:

- **Rollback:** Go back to previous known-good state

- **Redeploy:** Retry the same version again
-

Q2: How do you enable rollback support in CI/CD? A2:

- Retain previous artifacts
 - Use versioned releases
 - Maintain Git tags for stable builds
 - Use Blue-Green or Canary architecture
-

Q3: What's the role of monitoring in rollbacks? A3: Alerting systems (e.g., Datadog, CloudWatch) detect failures and can trigger automatic rollback or rollback approval.

Q4: Can CI/CD handle automatic rollbacks? A4:  Yes — if integrated with monitoring or error thresholds using tools like Argo Rollouts, Spinnaker, or custom logic.

◆ 12. Multi-Env Setup

Summary:

Multi-environment setup uses the same pipeline logic to deploy to multiple stages: `dev → staging → production`, often with different configs and approval flows.

Example:

```
- step:
  name: Deploy to Staging
  deployment: staging
  script: ./deploy.sh --env=staging
- step:
  name: Deploy to Production
  trigger: manual
  deployment: production
  script: ./deploy.sh --env=prod
```

Interview Q&A:

Q1: Why use multiple environments in CI/CD? A1: To safely validate changes in dev/staging before releasing to production — reduces risk and enables QA/testing.

Q2: How do you manage config differences across environments? A2:

- Use env vars or `.env` files
 - Use templating engines (e.g., Helm for K8s)
 - Load secrets/configs based on pipeline stage
-

Q3: Can environment promotions be gated? A3:  Yes — most tools support **manual approvals**, **merge constraints**, or **change windows**.

Q4: What's a good practice for managing environments? A4:

- Automate dev/test
 - Gate production with approvals
 - Keep environments consistent via IaC
-
-

13. Notifications

Summary:

Notifications in CI/CD keep your team informed of build status, failures, approvals, or deployments via tools like **Slack**, **email**, **Teams**, or **webhooks**.

Example: GitHub Actions Slack Notification

```
- name: Notify Slack
  uses: 8398a7/action-slack@v3
```

```
with:
  status: ${ job.status }
env:
  SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

Interview Q&A:

Q1: Why are CI/CD notifications important? A1: Provide real-time visibility into build, test, and deploy status. Helps teams respond quickly to failures or approvals.

Q2: How can you send notifications from Bitbucket Pipelines? A2:

- Use `curl` in script steps
 - Integrate with Slack via webhooks
 - Use third-party pipes like `atlassian/slack-notify`
-

Q3: What's a common problem with too many notifications? A3: Notification fatigue — make alerts meaningful by filtering for failed builds, deploys, or gated stages only.

Q4: Can CI/CD pipelines block based on approvals? A4:  Yes — most CI/CD tools support manual approval gates before notifying or continuing deployment to prod.

14. Caching in Pipelines

Summary:

Caching stores artifacts (like `node_modules`, Docker layers, etc.) between builds to speed up future runs.

Bitbucket Example:

```
caches:
  - node
```

GitHub Actions Example:

```
- uses: actions/cache@v3
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
```

Interview Q&A:

Q1: What can be cached in CI/CD? A1:

- Dependencies (`.m2` , `.npm` , `venv`)
 - Docker image layers
 - Build outputs
 - Compiled assets
-

Q2: What are the benefits of caching? A2:

- Faster builds
 - Reduced bandwidth
 - Lower compute costs
-

Q3: How do cache keys work? A3: A **cache key** identifies the version of cache. A hash of lock files ensures the cache is only reused when deps haven't changed.

Q4: When can caching go wrong? A4:

- Stale/incompatible cache
 - Conflicts between pipelines
 - Cache not being saved due to path/key mismatch
-

◆ 15. Secrets Manager

Summary:

Secrets Managers securely store and inject sensitive data like tokens, passwords, and keys into CI/CD pipelines **without hardcoding** them.

Tools:

- GitHub Secrets
 - Jenkins Credentials
 - AWS Secrets Manager
 - HashiCorp Vault
 - Doppler, 1Password CLI, etc.
-

Interview Q&A:

Q1: Why shouldn't secrets be in code? A1: It exposes sensitive data and poses a massive security risk (leaked tokens, credentials).

Q2: How are secrets injected at runtime? A2: Via environment variables or mounted files — securely fetched during the job.

Q3: How do you rotate secrets safely in CI/CD? A3:

- Use TTLs
 - Use automation to update secrets across tools
 - Re-deploy applications with the new secrets
-

Q4: What's the downside of using plaintext `ENV` vars for secrets? A4: They may be accidentally logged or dumped — use vault integration or mask variables in logs.

16. Release Tags

Summary:

Release tags are Git references (e.g., `v1.0.3`) that point to a specific commit and are often used to trigger release pipelines or define artifact versions.

Example:

```
git tag v1.0.3
git push origin v1.0.3
```

GitHub Actions:

```
on:
  push:
    tags:
      - 'v*'
```

Interview Q&A:

Q1: Why are Git tags useful in CI/CD? A1: Tags define stable points in history for releases, rollback, and deployment tracking.

Q2: How are tags used in Docker image versioning? A2: Tags map directly to image tags:

```
docker build -t myapp:v1.0.3 .
```

Q3: Can you auto-create tags in pipelines? A3:  Yes — with tools like `semantic-release` , or manually using Git CLI + CI environment variables.

Q4: What's the difference between lightweight and annotated tags? A4:

- **Lightweight:** Pointer to a commit
 - **Annotated:** Metadata + signature (preferred for releases)
-

◆ 17. Caching Strategies

Summary:

Caching strategy defines what to cache, when to reuse cache, and how to invalidate it. A smart strategy can cut build time by 50–90%.

Types of Caches:

Type	Example
Dependency Cache	<code>.npm</code> , <code>.m2</code> , <code>venv</code>
Docker Layers	Base images and build layer reuse
Build Outputs	<code>dist/</code> , <code>target/</code> , <code>.next/</code> , etc.
Compiler Cache	<code>ccache</code> , <code>Bazel</code>

Interview Q&A:

Q1: When does caching hurt instead of help? A1:

- When stale/invalid caches break the build
 - When large caches increase restore time
 - When cache keys are not unique enough
-

Q2: How to invalidate or bust a cache? A2:

- Change the cache key (e.g., include hash of `package-lock.json`)
 - Add manual cache versioning (e.g., `v2-node-${{ hashFiles(...) }}`)
-

Q3: Should you cache test reports or coverage? A3:  Sometimes — for long test setups or for reuse in artifact-based pipelines.

Q4: Difference between save vs restore cache in GitHub Actions? A4:

- **Restore:** Tries to fetch an existing cache by key
 - **Save:** Stores cache at end of job if key wasn't matched
-

◆ 18. Secrets Rotation

Summary:

Secrets rotation ensures that sensitive credentials (e.g., tokens, keys) are **periodically updated** and do not stay static or compromised.

Examples:

- AWS Access Keys → IAM Role + short-lived tokens (STS)
 - GitHub PAT → regenerate and update Secrets
 - Vault → dynamic credentials with TTL
-

Interview Q&A:

Q1: Why is secret rotation critical? A1: To prevent misuse if leaked, comply with security policies, and minimize blast radius.

Q2: How can you rotate secrets in CI/CD? A2:

- Use a secrets manager that supports rotation (e.g., AWS Secrets Manager)
 - Integrate secret fetch during runtime instead of hardcoding
 - Re-deploy pipelines/apps when credentials are updated
-

Q3: Can CI/CD systems support auto-rotating secrets? A3:  Yes — if integrated with secret backends like Vault, Doppler, or AWS Secrets Manager.

Q4: How to detect expired or rotated secrets? A4:

- Monitor failed builds due to auth errors
 - Use secret scanners or expiry alerts
 - Include pre-checks in pipeline health step
-

◆ 20. Environment Setup

🔍 Summary:

This covers initializing and preparing each pipeline environment, including OS-level tools, programming runtimes, and any global installs.

✅ Example: GitHub Action Setup

```
- name: Setup Node.js
  uses: actions/setup-node@v3
  with:
    node-version: 18
```

💬 Interview Q&A:

Q1: How do CI/CD runners get provisioned? A1: Cloud-hosted runners spin up fresh VMs or containers, usually on Linux or Windows base images.

Q2: What are common setup tasks? A2:

- Language runtime (Python, Node, Java)
 - Tooling (`aws-cli` , `kubectl`)
 - Dependency installation (`apt-get` , `npm` , `pip`)
-

Q3: How do you debug failures in setup step? A3: Add verbose logging (`set -x`), isolate failing step, and re-run locally or in debug mode (e.g., self-hosted runner).

Q4: Can you reuse setup logic across projects? A4:  Yes — via reusable workflow templates, scripts, Docker base images, or pre-configured runners.

Bonus 5 CI/CD Concepts (Must-Have, Often Missed)

◆ 1. Canary Deployment Pipelines

 **What:** Gradual rollout to a small % of users (e.g., 5%, then 25%, then 100%).

 **Why important:** Safer than full rollout; lets you rollback quickly if metrics (e.g., error rate) go bad.

 **Interview Tip:** Pair with **monitoring tools** (Datadog, Prometheus, CloudWatch alarms).

 **Sample (Kubernetes + Argo Rollouts):**

```
strategy:
  canary:
    steps:
      - setWeight: 10
      - pause: {duration: 1m}
      - setWeight: 50
      - pause: {duration: 2m}
```

◆ 2. Matrix Builds

 **What:** Run your pipeline across **multiple environments/configs** (e.g., Node 14, 16, 18).

 **Why important:** Detect compatibility issues early, especially for SDKs, APIs, or OSS projects.

 **GitHub Actions Example:**

```
strategy:
  matrix:
    node-version: [14, 16, 18]
```

```
steps:
  - uses: actions/setup-node@v3
    with:
      node-version: ${ matrix.node-version }
```

◆ 3. Artifact Management

📌 **What:** Store pipeline outputs like `.whl` , `.jar` , `.zip` , or Docker images.

✅ **Why important:** Enables reproducibility, rollback, and CD across environments.

💡 **Bitbucket Example:**

```
artifacts:
  - dist/**
```

GitHub:

```
- uses: actions/upload-artifact@v3
  with:
    name: build-dist
    path: dist/
```

◆ 4. Code Coverage & Quality Gates

📌 **What:** Automatically **fail pipeline** if coverage/test/lint doesn't meet thresholds.

✅ **Tools:**

- **Coverage:** `pytest-cov` , `jest --coverage` , `nyc`
- **Quality Gates:** SonarCloud, Codecov, DeepSource

💡 **GitHub Action:**

```
- run: pytest --cov=myapp
```

- uses: `codecov/codecov-action@v4`

◆ 5. Supply Chain Security (SBOM / SLSA)

📌 **What:** Prevent compromised dependencies or tampered builds.

✅ **Why important:** SolarWinds & Log4j attacks were supply chain failures.

📦 **Techniques:**

- Use `npm audit`, `pip-audit`, `trivy`
 - Generate SBOM (Software Bill of Materials): CycloneDX, Syft
 - Adopt SLSA framework (<https://slsa.dev>)
-